プログラム合成対象言語 Pro5Lang における知識表現形式

第27回 人工知能学会 汎用人工知能研究会(SIG-AGI)

一杉裕志,高橋直人,竹内泉（産総研）,
佐野崇（東洋大学）,中田秀基(順天堂大学)

2024-08-08

私の研究の中期的目標

[一杉+ 第18回 汎用人工知能研究会(SIG-AGI), 2021]

- **実世界を極力単純化した環境**で
  脳型AGIのデモを動かす
- 中核技術
  - プログラム合成対象言語 Pro5Lang
    [一杉+ 第20回 汎用人工知能研究会, 2022]
  - 再帰的強化学習 RGoal
    [Ichisugi+ 2019]
  - 大脳皮質モデル BESOM
    [Ichisugi 2007]

今日の話

脳型AGIアーキテクチャの全体像については、
今日の発表内容でほぼ固まった
（今後は？）

本研究の背景と概要

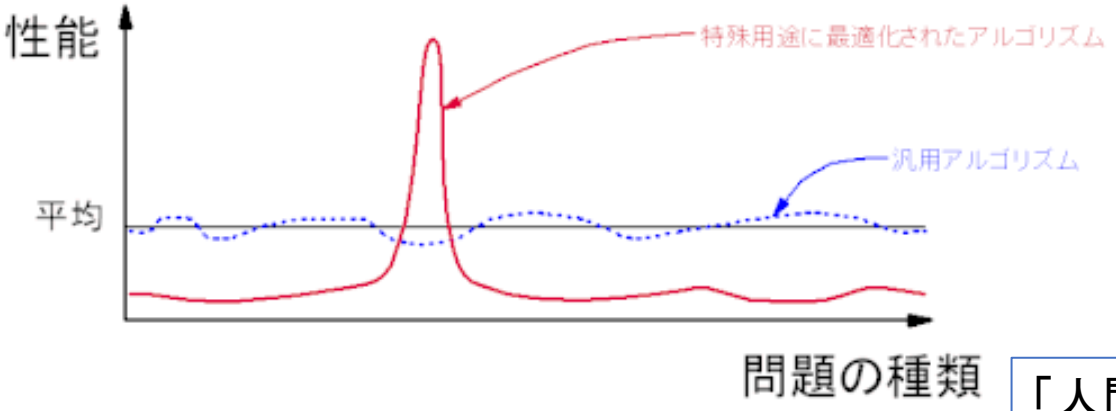
- プログラム合成はＡＧＩ実現に向けた有望なアプローチの１つ
  - 自分自身の経験から報酬を最大化するプログラムを自律的に合成
- プログラム合成の性能は合成対象言語の言語仕様に左右される
- これまで主に合成対象言語 Pro5Lang の制御構造について設計してきたが、データ構造の表現方法に関する考察は不十分であった
- 本研究では、ヒトの脳にヒントを得て、高い表現力とプログラム合成のしやすさを兼ね備えた知識表現形式の設計

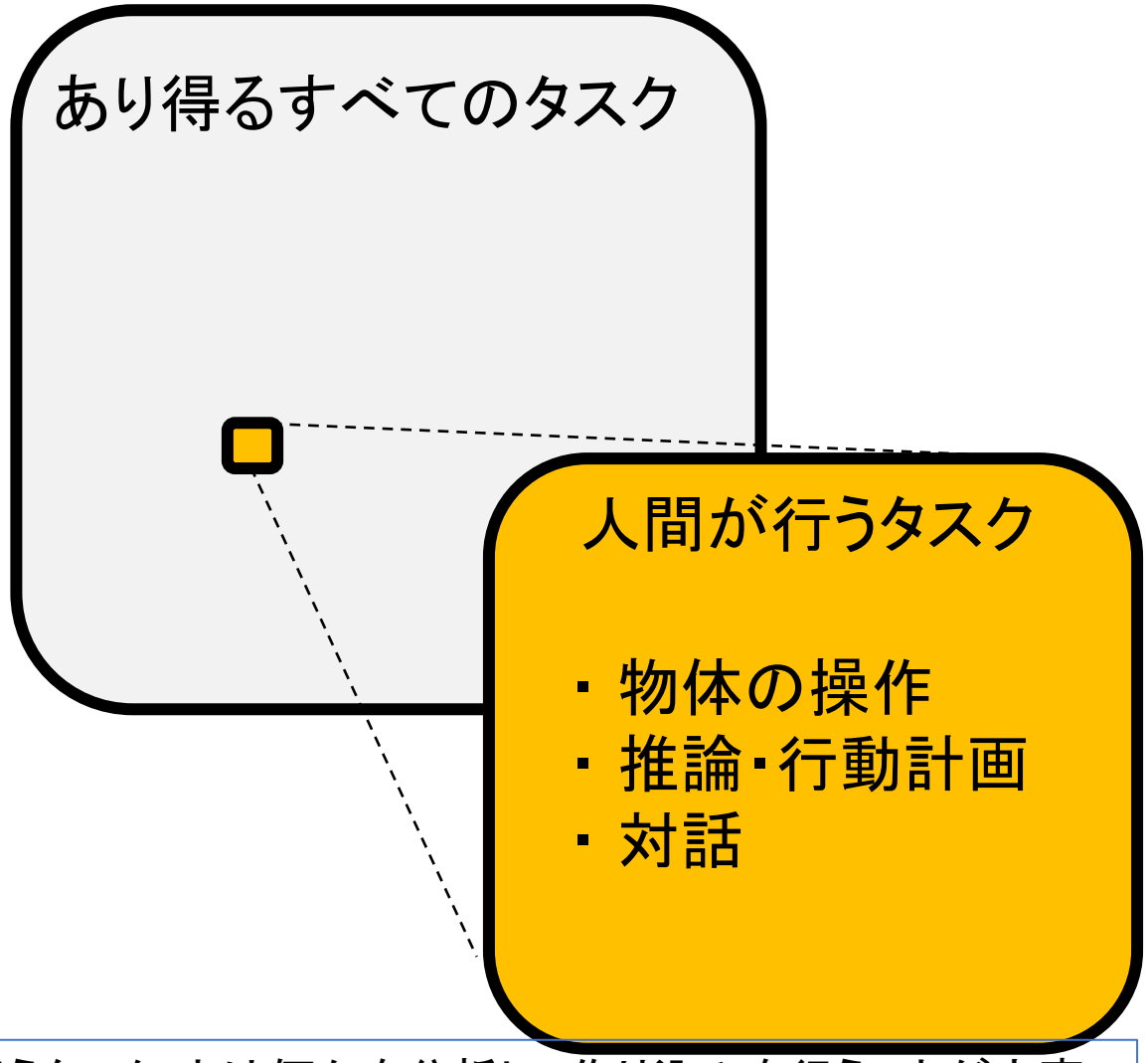
- 現状の大規模言語モデル(LLM)の問題点についても考察

計算効率・汎化性能

性能を上げるには特殊な作り込みが必要

- ノーフリーランチ定理
  [David H.Wolpert and William G. Macready 1995]
- 「工学者や最適化の専門家にとって、この定理は（中略）領域を限定して特殊な最適化ルーチンを作成すべきであることを示している。」
  「ノーフリーランチ定理 - Wikipedia」

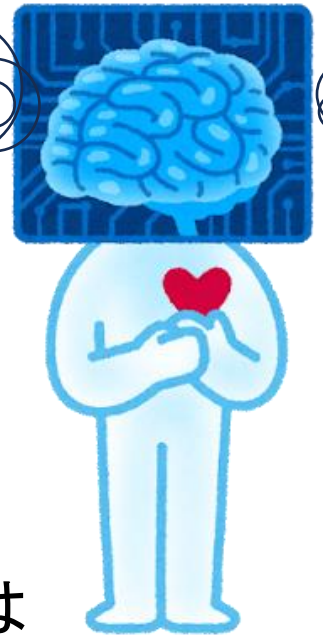




「人間が行うタスク」とは何かを分析し、作り込みを行うことが大事
（ただし**多くのAGI研究者はそういうアプローチをとっていない**）

社会のルール

他者の心理



数学や自然科学の法則

汎用人工知能(AGI)は
社会のルール、他者の心理、数学や自然科学の法則
などに関する複雑な知識も表現・処理できる必要がある

「かわいいフリー素材集 いらすとや」
https://www.irasutoya.com/

合成対象言語 Pro5Lang

- 機械語と論理型言語の特徴を合わせ持った手続き型言語
- **機械語に似た特徴**：
  - 固定長のデータ、固定長の命令
  - 少数のレジスタと大容量のメモリ
- **論理型言語に似た特徴**：
  - パターンマッチングによる行動ルール選択
  - （学習が理想的に進めば）正しい推論をする
- 論理型言語 Prolog との違い：
  - Pro5Lang の基盤は数理論理学ではなく強化学習
  - 確率、価値
  - 矛盾を許容、間違った推論を許容、値〈unknown〉を許容、例外を許容
  - 非決定論的動作（Cf. MCTS モンテカルロ木探索）

```
 1: k(e(0, 0, Socrates, Is, AMan, 0));      // ソクラテスは人間である
 2: k(e(If, c(0, 0, x, Is, AMan, 0), c(0, 0, x, Is, Mortal, 0))); // 人間は死ぬ

 3: eifxy = e(If, c(x1,x2,x3,x4,x5,x6), c(y1,y2,y3,y4,y5,y6));
 4: eif0y = e(If, c(__,__,__,__,__,__), c(y1,y2,y3,y4,y5,y6));
 5: ax = a(x1,x2,x3,x4,x5,x6);
 6: ay = a(y1,y2,y3,y4,y5,y6);
 7: rule(w(),       w(ay), recall(eif0y));  // y を証明するために x → y を想起
 8: rule(w(eifxy),  w(ay), call(ax));       // x → y なら y を証明するために x を証明
 9: rule(w(ax),     w(ay), recall(eifxy));  // x なら y を証明するために x → y を想起
10: rule(w(ax, eifxy), w(ay), set(ay));     // x, x → y なら y が成り立つ

11: ex = e(x1,x2,x3,x4,x5,x6);
12: rule(w(),   w(ax), recall(ex));
13: rule(w(ex), w(ax), set(ax));
```

プロトタイプ版 Pro5Lang のプログラム例
（シンボルのベクトルに変換されて処理される）

知識表現形式の設計に参考にした過去の技術

- 状況理論・状況意味論
- 自然演繹

- その他
  - 記号AIにおける状況計算
  - RDF (Resource Description Framework)
  - 認知科学における状況モデル
  - 数理論理学における可能世界意味論
  - Lisp, Prolog, Java などのプログラミング言語など

状況理論・状況意味論［Barwise and Perry 1983］

The man I saw running yesterday is at the door,

「きのう走っているのを私が見た男が、ドアにいる」

she is making use of a situation that she witnessed the day before, the one in which a certain man was running, in order to identify the man at the door.

There is another situation, r, a situation that occurred the day before the utterance, and which Melissa witnessed, such that

話者が
話している状況

$$
\begin{aligned}
u \models\ & \langle\langle \text{utters, Melissa, } \Phi, l, t, 1 \rangle\rangle \wedge \\
& \langle\langle \text{refers-to, Melissa, } \textit{the man}, M, l, t, 1 \rangle\rangle \wedge \\
& \langle\langle \text{refers-to, Melissa, } \textit{the door}, D, l, t, 1 \rangle\rangle
\end{aligned}
$$

同一人物を
同じシンボルで表現

where Φ is the sentence

The man I saw running yesterday is at the door

and where Melissa is making use of r and the fact that M is the unique man such that (for some appropriate values of l', t')

きのうの状況

$$r \models \langle\langle \text{runs}, M, l', t', 1 \rangle\rangle$$

[Devlin 2006] p.12

数理論理学における自然演繹

- 自然演繹： Gentzen と Jaśkowski が独立に開発(1934)
- 人間が行う証明のように、恣意的で一時的な仮定の導入が可能
  - 間接証明（対偶の証明、背理法など）、場合分けなど

$$
\begin{array}{rl}
\dfrac{[p]^1 \qquad [p \to q]^3}{q} & ass. \\[2ex]
\dfrac{q \qquad\qquad [q \to r]^2}{r} & (\to E) \\[2ex]
\dfrac{r}{p \to r^1} & (\to E) \\[2ex]
\dfrac{p \to r^1}{(q \to r) \to (p \to r)^2} & (\to I) \\[2ex]
\dfrac{(q \to r) \to (p \to r)^2}{(p \to q) \to ((q \to r) \to (p \to r))^3} & (\to I) \\[2ex]
 & (\to I)
\end{array}
$$

証明の例

参考:
「Natural deduction - Wikipedia」
https://en.wikipedia.org/wiki/Natural_deduction
「Natural Deduction | Internet Encyclopedia of Philosophy」
https://iep.utm.edu/natural-deduction/

シークエント計算 [Gentzen 1936]

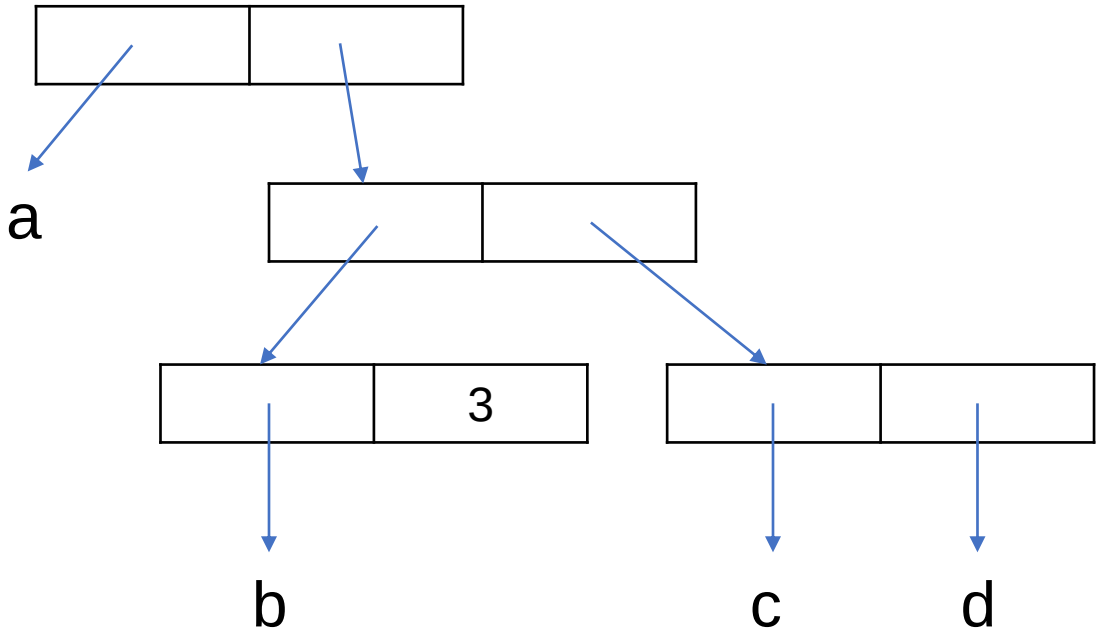
- シークエント「仮定の集合 ⊢ 結論」に対して推論規則を適用
- 単純だが表現力の高い強力な枠組み
- 「仮定の集合」は状況意味論「状況」に近い

$$
\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{p \vdash p \qquad p \to q \vdash p \to q}{p, p \to q \vdash q} \qquad q \to r \vdash q \to r}{p, p \to q, q \to r \vdash r}}{p \to q, q \to r \vdash p \to r}}{p \to q \vdash (q \to r) \to (p \to r)}}{\vdash (p \to q) \to ((q \to r) \to (p \to r))}
$$

「Natural Deduction | Internet Encyclopedia of Philosophy」
https://iep.utm.edu/natural-deduction/

Lisp におけるシンボルとポインタ

(a . ((b . 3) . (c . d)))

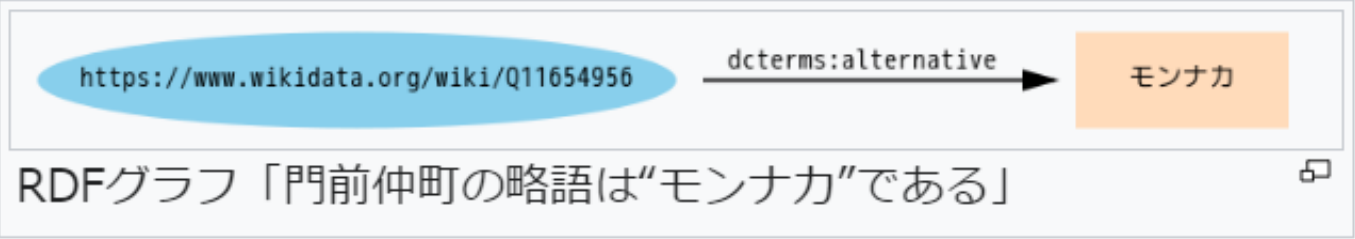


・cons cell は固定長（長さ2）のデータ構造
・ポインタは cons cell やシンボルなどを参照
・実装によっては整数は cons cell に直接埋め込まれる

これだけで複雑なデータ構造が表現可能

RDF (Resource Description Framework)

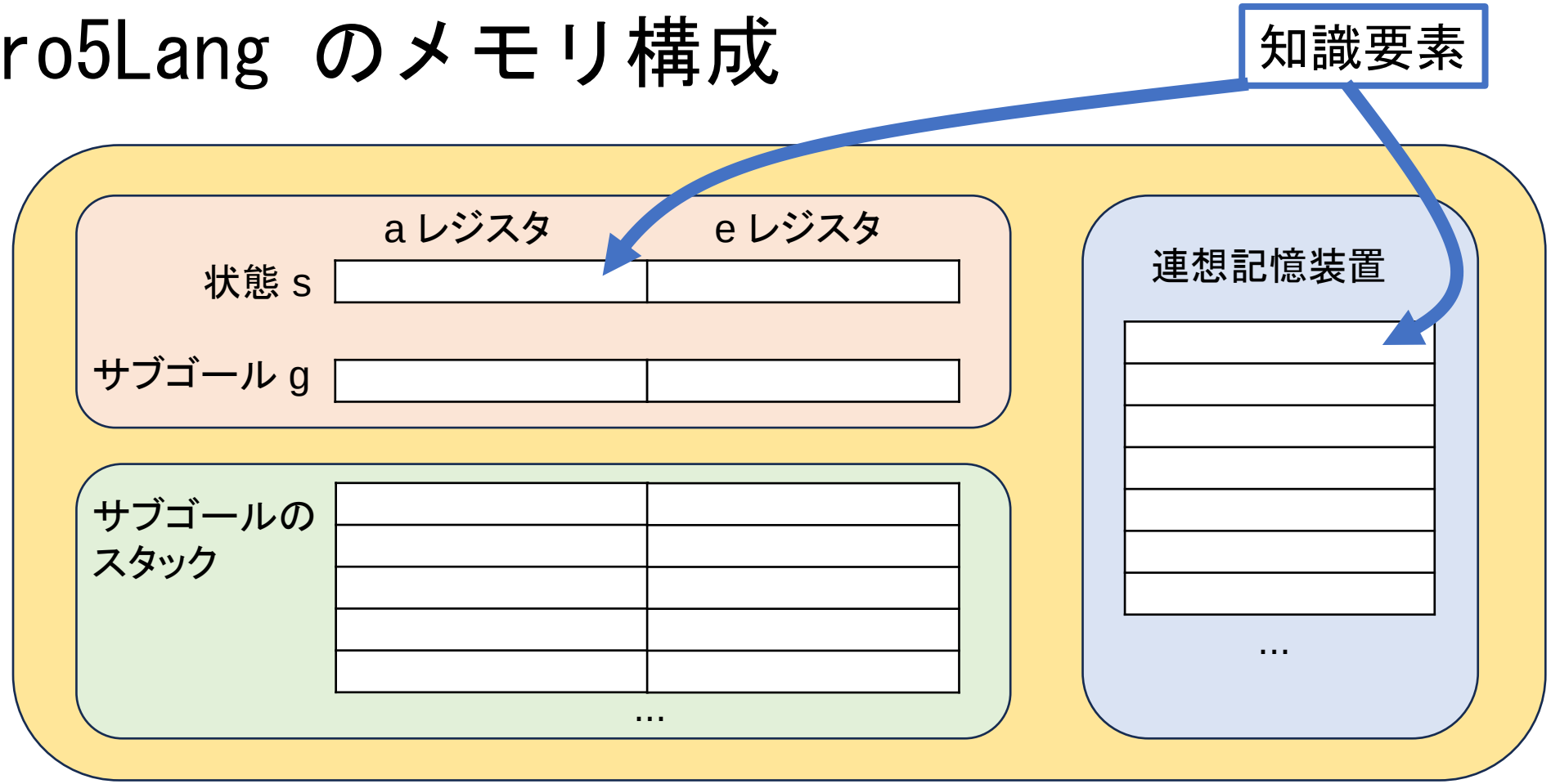
- インターネット上のメタデータを表現するために定義された知識表現形式
- 三つ組み（主語、述語、目的語）が基本単位
- ノードは、シンボルではなく URI で表現



「Resource Description Framework - Wikipedia」
https://ja.wikipedia.org/wiki/Resource_Description_Framework

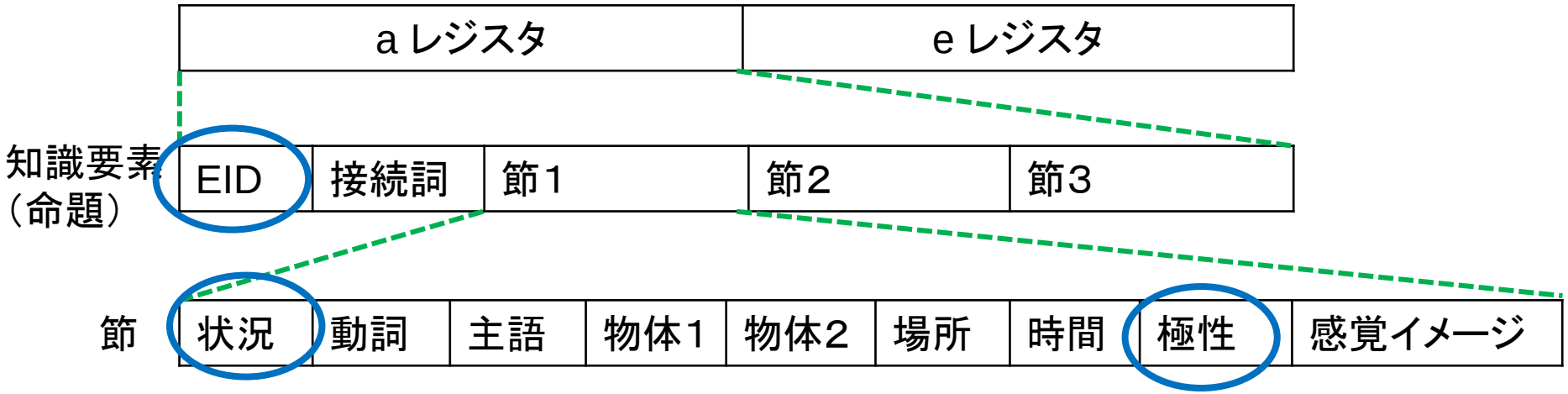
Pro5Lang の知識表現形式

Pro5Lang のメモリ構成



・レジスタの値、連想記憶装置の値はすべて「知識要素」と呼ぶ固定長ベクトル
・(s, g) の値によって行動ルールを選択し実行
・推論結果は a レジスタに入れると同時に、自動的に連想記憶装置に追加

情報処理の基本単位：知識要素(Knowledge element)



1つの知識要素は複文程度の情報を表現できる

注：
EID 、感覚イメージは未実装
節には「様相」のスロットを追加予定

極性(Polarity)

- 極性(Polarity)は値 true か false で、節の内容が**否定文**か**肯定文**かを表す
  - 「きのう学校にネコがいた」
    (Actual, Cat, exists, …. School, Yesterday, **true**)
  - 「きのう学校にネコがいなかった」
    (Actual, Cat, exists, …. School, Yesterday, **false**)

- 極性に ″+″ を指定することでサブゴールに whether を表現できる
  - 「きのう学校にネコがいたかどうか」を推論するサブルーチン呼び出し：
    call(<..., _, Simple, Actual, Cat, exists, …. School, Yesterday, **+**, …>)

状況

- Pro5Lang の「状況」は、
  知識要素（命題）の中の節の真偽値を決めるために必要な情報
- 「火星人が地球を攻めてきた」
  - 現実世界では 偽
  - 小説「宇宙戦争」の中では 真
  - ラジオドラマ「宇宙戦争」を信じた人の心の中では 真
- 参考：状況意味論 [Barwise and Perry 1983] [Devlin 2006] での用語：
  - 状況 situation s
  - インフォン infon σ
  - 命題 proposition $s \models \sigma$
    (s supports σ と読む)

状況の継承

- 状況に継承関係を持たせる利点：
  - 記憶域の節約
  - 強化学習のサンプル効率の向上
  - 知識の部分的変更が容易
- 状況 s を ss が継承するとき、デフォルトでは、
  s がサポートするσを ss もサポートする
  Inherit(s,ss),s ⊨ σ ⊢ ss ⊨ σ
- Pro5Lang では継承の処理の方法に３種類ある
  1. プログラムで明示的に処理する方法
  2. 行動ルールの変数を使う方法
  3. 状況を表す特徴ベクトルのパターンを使う方法（未実装）

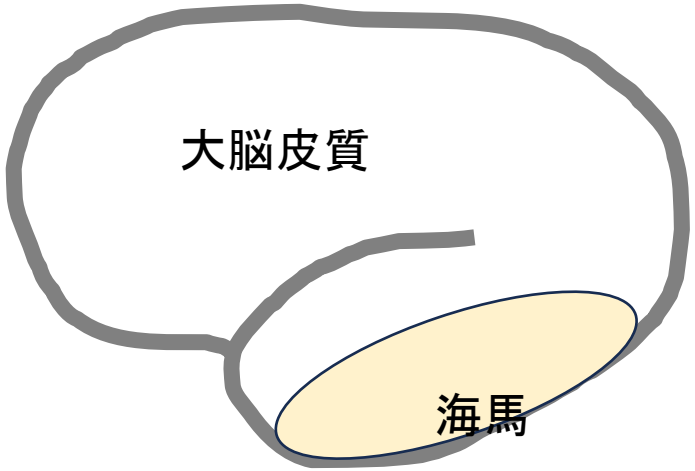
シンボルとポインタ

- 過去のAI関連技術を見る限り、複雑な情報を表現するためにシンボルとポインタのようなものは不可欠に思える
- 問題点：神経科学的に妥当なポインタの実現方法は？

- 「ネコ」とか「走る」などの抽象概念は、ニューラルネットワークで大量のデータから学習可能
- 問題点：「いま目の前にあるコップ」のような一時的な概念を表すシンボルは？

「かわいいフリー素材集 いらすとや」
https://www.irasutoya.com/

脳における記憶の固定化



即時記憶：大脳皮質のニューロン発火

↓ 瞬時

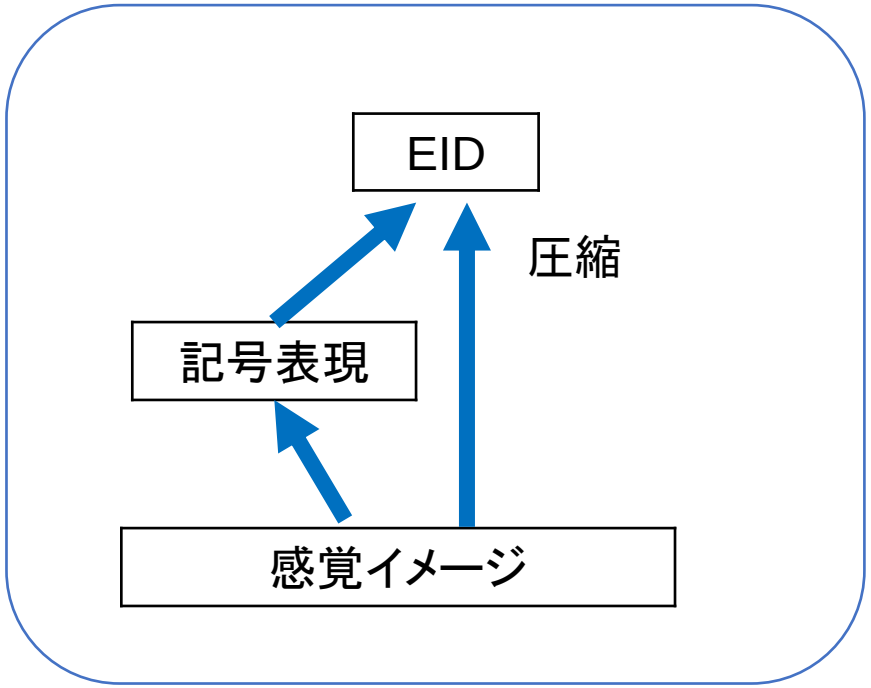
近時記憶：海馬シナプス

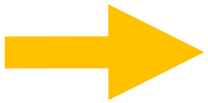
↓ 数週間から数年

遠隔記憶：大脳皮質シナプス

記憶場所が2回変わる。
→ **皮質や海馬のニューロンの「アドレス」をポインタとして使うとは考えにくい**

知識要素ID（EID）は知識要素の中身を圧縮したもの





知識要素

EID	記号表現	感覚イメージ

・大脳皮質のオートエンコーダーで圧縮することを想定
・EIDは
　C 言語におけるポインタ
　ハッシュ表におけるハッシュ値
　自然言語処理における embedding
　記号AIにおけるシンボル

シンボル ＝ EID ＝ ポインタ

知識要素
（命題）

EID	接続詞	節１	節２	節３

節

状況	動詞	主語	物体１	物体２	場所	時間	極性	感覚イメージ

「白い犬が黒い猫を追いかけた」

連想記憶装置

ID1	-	白い犬を見た	-	-
ID2	-	黒い猫を見た	-	-
ID3	-		-	-

…

抽象概念を wild card を使った特徴ベクトルで表現可能

		（足、毛、キバ、たてがみ、大きさ）
けもの	=	（４本、あり、　＊、　＊、　＊）
猛獣	=	（４本、あり、あり、　＊、　大）
ライオン	=	（４本、あり、あり、あり、　大）
牛	=	（４本、あり、なし、なし、　大）

「猛獣は肉を食べる」という宣言的知識を持っているとき、
recall(「ライオンは〈＋〉を食べる」)というクエリで
「ライオンは肉を食べる」という命題が想起できる。

EID に関して起こり得る問題

- ＩＤの衝突
  - 見た目で区別付かない２つの物体や人物の区別、よく似た２つのエピソードの区別など
- ＩＤの変化
  - 知識が増えることによって対象の概念の特徴ベクトルが変化する場合など
- ＩＤの統合
  - 異なる概念だと思っていたものが実は同一のものであると知ったときなど
- ＩＤの分離
  - 同じものだと信じていたものが別のものだとわかったときなど

すべて経験で獲得した Pro5Lang プログラムが対処する！
（RGoal の性質により、のちのち役立つように脳内の知識を整理するようなプログラムが自律的に獲得されるはず。）

例：見た目が同じコップ

- 同じ特徴の物体が複数ある時にはのちのち情報処理が混乱することを予見し、対策を講じるはず
- 具体的には、**「自分の右側にあるコップ」とか「水が半分になったコップ」**のように、あとで判別可能な特徴を能動的に付与し、異なる感覚イメージを持たせることで、圧縮した特徴ベクトルも異なるものにできるはず

「かわいいフリー素材集 いらすとや」
https://www.irasutoya.com/

テストプログラムの実装

- 命題に「状況」を導入した結果、仮想的状況における知識と、現実の状況における知識の両方を用いた推論が可能になるはず
- Pro5Lang のプロトタイプ実装の上で、
簡単なテスト用タスクを解くプログラムを手で書いて動作確認
  - 背理法による証明の例
  - 場合分けによる証明の例

背理法による証明の例

- 弟が「おかしを食べてない」と言っているが、実際には食べたことを証明したい。
- ゴミ箱におかしのゴミがあるのを見たとする。
- 「おかしを食べてないならばゴミはない」という知識があるものとする。
- 「もしおかしを食べてないならゴミがないはずだが、ゴミを見たので矛盾する。だから実際は食べたはずだ。」

「嬉しそうにチョコレートを持っている人のイラスト（男性）| かわいいフリー素材集 いらすとや」
https://www.irasutoya.com/2016/12/blog-post_40.html

```
// 背理法 (proof by contradiction) で g を証明するサブルーチン
pbcg = PbyC(s ⊨ g)
rule(_, pbcg, call(Assumed(s, +, not(g))))
rule(Assumed(s, ss, not(g)), pbcg, set(ss ⊨ not(g)))
rule(ss ⊨ not(g), pbcg, call(ss ⊨ Contradiction)) // 矛盾が起きることを証明
rule(ss ⊨ Contradiction, pbcg, recall(Assumed(s, ss, not(g))))
rule({ss ⊨ Contradiction, Assumed(s, ss, not(g))}, pbcg, set(PbyC(s ⊨ g)))

// s を継承する状況 ss で節 c を仮定するサブルーチン
ag = Assumed(s, +, c)
rule(_, ag, primitive(NewSituation,s))
rule(Inherits(s, ss), ag, set(ss ⊨ c)) // 新規に作った状況 ss で c を仮定
rule(ss ⊨ c, ag, recall(Inherits(s, ss)))
rule({ss ⊨ c, Inherits(s, ss)}, ag, set(Assumed(s, ss, c)))
```

図 5: 背理法で証明する Pro5Lang プログラムの疑似コード。小文字で始まる記号は変数、大文字で始まる記号は定数。“_” は任意の値に、“+” は unknown 以外の値にマッチするワイルドカード。

```
// 任意の状況 ss で ss ⊨ not(A) ならば ss ⊨ not(G) が成り立つという宣言的知識
k(<If, ss ⊨ not(A), ss ⊨ not(G)>)

// ss のもとで矛盾が生じることを証明（PbyC から呼ばれる）
cg = ss ⊨ Contradiction
rule(ss ⊨ not(A), cg, recall(<If, ss ⊨ not(A), ss ⊨ +>))
rule({ss ⊨ not(A), <If, ss ⊨ not(A), ss ⊨ not(G)> }, cg, set(ss ⊨ not(G)))
rule(ss ⊨ not(G), cg, call(ss ⊨ G))
rule(ss ⊨ G, cg, recall(ss ⊨ not(G)))
rule({ss ⊨ G, ss ⊨ not(G)}, cg, set(ss ⊨ Contradiction))

// 「Actual （現実の状況）において G が成り立つ」を意味する宣言的知識
k(Actual ⊨ G)

// 状況 ss において G が成り立つことを証明するサブルーチン
ssgg = ss ⊨ G
rule(_, ssgg, recall(Inherits(+, ss)))
rule(Inherits(s, ss), ssgg, recall(s ⊨ G))
rule({Inherits(s, ss), s ⊨ G}, ssgg, set(ss ⊨ G))

// タスクのゴール：背理法により A が成り立つことを証明
taskGoal(PbyC(Actual ⊨ A))
```

状況の
継承処理

図 6: 背理法で命題 A を証明する Pro5Lang プログラムの疑似コード。Actual: 現実の状況、A: 「おかしを食べた」、 G: 「おかしのゴミがある」。

場合分けによる証明の例

- あした学校にいく必要があることを証明したい。
  - あした雨なら映画観賞会がある。
  - 映画観賞会があるなら学校に行く必要がある。
  - あした雨でないなら運動会がある。
  - 運動会があるなら学校に行く必要がある。

「バトンを渡す人のイラスト（運動会）| かわいいフリー素材集 いらすとや」https://www.irasutoya.com/2016/03/blog-post_823.html
「映画館のイラスト | かわいいフリー素材集 いらすとや」https://www.irasutoya.com/2013/07/blog-post_7666.html

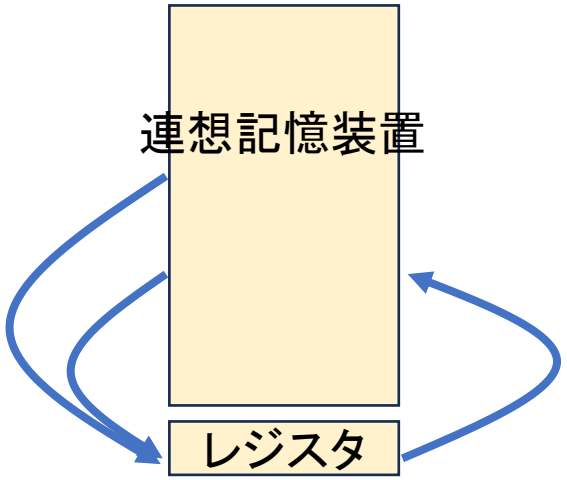
```
// 状況 s を継承する状況 ss において、
// c を仮定すると g が成り立つことを証明するサブルーチン
cgg = Case(s, +, c, g)
rule(_, cgg, call(Assumed(s, +, c)))
rule(Assumed(s,ss,c), cgg, set(ss ⊨ c))
rule(ss ⊨ c, cgg, call(ss ⊨ g))
rule(ss ⊨ g, cgg, recall(Assumed(s,ss,c)))
rule({ss ⊨ g, Assumed(s,ss,c)}, cgg, set(Case(s, ss, c, g)))

// 状況 s で S が成り立つことを、
// R の時と not(R) の時で場合分けして証明するサブルーチン
cg = s ⊨ S
rule(_, cg, call(Case(s, +, R,S)))
rule(Case(s, _, R,S), cg, call(Case(s, +, not(R),S)))
rule(Case(s, _, not(R),S), cg, recall(Case(s, +, R,S)))
rule({Case(s, _, not(R),S), Case(s, _, R,S)}, cg, set(s ⊨ S))

// S が成り立つことを証明するサブルーチン
sg = s ⊨ S
rule(s ⊨ R, sg, set(s ⊨ E1))
rule(s ⊨ E1, sg, set(s ⊨ S))
rule(s ⊨ not(R), sg, set(s ⊨ E2))
rule(s ⊨ E2, sg, set(s ⊨ S))

// タスクのゴール： S が成り立つことを証明
taskGoal(Actual ⊨ S)
```

図 7: 場合分けで命題 S を証明する Pro5Lang プログラムの疑似コード。S:「明日学校に行く必要がある」、 R:「明日は雨」、E1:「学校でイベント E1 がある」、E2:「学校でイベント E2 がある」。

Pro5Lang プログラムと LLM の動作の類似点

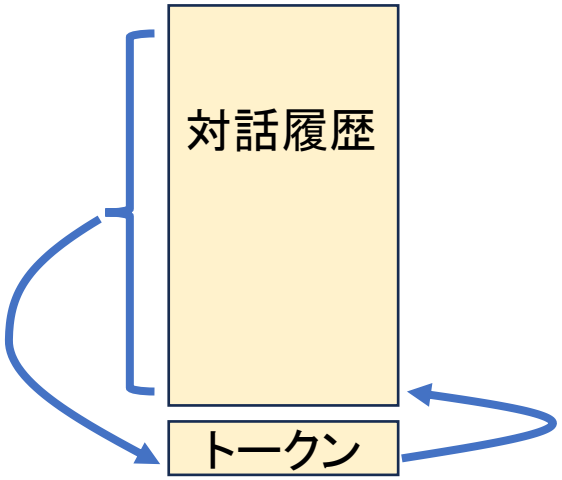
	Pro5Lang プログラム	LLM
各ステップの動作	新たな命題を推論	次のトークンを推論
プログラムの実体	学習で獲得した行動ルール集合	パラメタの値
メモリへの書き込み	推論結果の追加	推論結果の追加
メモリの読み出し	クエリによる検索	self-attention による検索

メモリの読み書きに制約があるため解の探索空間が狭くプログラム合成が容易



連想記憶装置内の命題から新たな命題を推論

推論した命題を連想記憶装置に追加



対話履歴からトークンを推論

推論したトークンを対話履歴に追加

知識表現方法の観点から見た現状の LLM の問題点

- 現状の LLM は対話履歴の形でしか「プログラムの内部状態」を持てない
  →
  - 知識の抽象化が不十分なので、知識の追加と検索が非効率的かつ不正確
  - トークン列で表現できない感覚イメージなどを直接表現できない

- 現状の LLM はプログラムとデータの表現が全く違う
  （プログラム＝パラメタの値、 データ＝トークン列）
  → Lisp 言語の eval や Prolog 言語の assert を使った
  「**自己書き換えコード**」のようなことがやりにくい

これらの問題が、ヒトレベルの知能への到達の障害になるのではないか？

まとめと今後

- プログラム合成対象言語 Pro5Lang の知識表現形式を提案
  - ヒトの脳内における知識表現方法の１つの仮説でもある
- 情報処理の基本単位は**知識要素**という固定長ベクトル
- **状況**を導入、背理法と場合分けを用いた推論を、テストプログラムで動作確認
- シンボルやポインタは **EID （知識要素 ID）**で実現可能（未実装）
- Pro5Lang を中核技術とする脳型 AGI アーキテクチャの全体像が、これでほぼ固まった

- Pro5Lang と LLM との類似点と相違点について考察
  - 現状の LLM のままのヒトレベルの到達は非現実的に思える

以上